DSA03 – NATURAL LANGUAGE PROCESSING

ASSIGNMENT – 1

BUILD A MORPHOLOGY-AWARE TEXT NORMALISATION AND SEARCH LAYER FOR A HOSPITAL RECORD SYSTEM

Submitted by: NASINA UMASRI

Roll Number: 192424240

Course: DSA03 – Natural Language Processing

Academic submission draft based on the uploaded assignment brief

DECLARATION AND SCOPE

This document answers all twelve tasks in the uploaded DSA03 Natural Language Processing Assignment–1. It follows the hospital-record scenario and the terminology used in the question paper: regular expressions, finite-state automata, inflectional and derivational morphology, finite-state morphological parsing, orthographic rules, Porter stemming, implementation, evaluation and the Day-4 design challenge.

The assignment requires a corpus of at least 500 de-identified records, 25,000 tokens, four record types, 12 entity patterns, four automata, a lexicon of at least 300 stems, 20 inflectional affixes, 15 derivational affixes, 40 clinician queries and 200 manually annotated word forms. The question paper does not provide those records or annotation results. Therefore, this answer supplies a reproducible synthetic-corpus design, executable examples, hand-worked traces and clearly marked illustrative evaluation tables. Illustrative values must not be represented as measurements from a real hospital.

For a final empirical submission, run the supplied implementation on the generated or approved de-identified corpus and replace every value marked illustrative with the actual program output. No real patient information should be entered into the example corpus.

The proposed system is a decision-support search layer only. It does not diagnose patients, recommend treatment or replace clinical judgement. Abbreviation expansion and approximate matching are presented as candidate normalisation operations and require clinical governance before deployment.

CONTENTS AND ANSWER MAP

Section	Coverage
Task 1	Problem framing; linguistic levels; model families; ambiguity; search; pipeline
Task 2	Regex operators and 12 clinical entity patterns; matching behaviour; error analysis
Task 3	Tokenizer; maximal munch; normalisation; irreversible loss; limits of regular expressions
Task 4	DFA/NFA definitions, validators, traces, subset construction, minimisation, equivalence, pumping lemma
Task 5	Inflectional morphology; paradigms; spelling rules; irregulars; type-to-lemma ratio
Task 6	Derivational morphology; 15+ affixes; families; non-compositionality; productivity; conflation
Task 7	Finite-state transducer; continuation classes; parsing and generation; ambiguity
Task 8	Orthographic rewrite rules; cascade; two-level morphology; tests
Task 9	Porter algorithm; worked examples; stemming vs lemmatisation; errors; retrieval
Task 10	Python implementation; tests; metrics; plots; reproducibility
Task 11	Comparison and deployment recommendation
Task 12	OCR and code-mixed input impact, evaluation, redesign and defence
Appendices	Synthetic corpus, annotation schema, implementation checklist and references

Recommended reading order: read the design assumptions first, then Tasks 1–12. Tables and code blocks are intended to be adapted to the actual corpus. A regex match means only that a string has the expected surface shape; it does not establish that the value is clinically correct.

EXECUTIVE SUMMARY

The hospital's search problem is caused by variation at several levels. A clinician may search for “fracture” while records contain “fractures”, “fractured” or “fracturing”. Identifiers, dates, strengths, measurements and abbreviations also appear in inconsistent formats. A search layer must therefore recognise structured entities, split text into useful tokens, normalise predictable spelling variation and connect surface word forms to lexical stems and grammatical features.

The recommended architecture is a hybrid finite-state pipeline. Regular expressions recognise structured patterns and perform deterministic text cleanup. Deterministic finite automata validate common identifiers and numeric formats. A finite-state transducer maps lexical stems plus features to surface forms and back. A Porter stemmer is retained only as a fallback for unknown English words, never as the sole clinical normaliser. A weighted edit-distance stage and token-level language identification are added for OCR noise and code-mixed notes.

The key design principle is conservative conflation. Inflectional forms such as fracture/fractures can normally be connected. Derivational relatives such as renal/nephrectomy should be linked through curated semantic relations rather than blindly reduced to the same stem. Clinically distinct terms must not be merged merely because their character endings look similar.

The Day-4 requirement changes the input assumptions. OCR introduces character substitutions and line-break errors, while Romanised regional-language notes introduce out-of-lexicon tokens and non-English morphology. Exact regular expressions and English-only lexicons will lose coverage. The design should therefore be modified, not discarded: retain the reliable finite-state core, add a noise-tolerant front end, extend curated multilingual resources and route uncertain cases for review.

TASK 1 – FRAME THE PROBLEM

Models, algorithms and levels of analysis

1.1 Problem definition. The input is a collection of free-text hospital documents: discharge summaries, prescriptions, operation notes and nursing observations. The output is a searchable, normalised representation that preserves the original text while adding tokens, recognised entities, lexical analyses and canonical search keys. The system must improve recall without creating unsafe matches.

The central challenge is not simply spelling correction. A string such as "500mg" contains a quantity and a unit; "BP 120/80 mmHg" contains a structured measurement; "fractures" is an inflected noun; "nephrectomies" combines a clinical derivational base and plural inflection; and "renal" is a related clinical term but not an inflectional form of nephrectomy. Different linguistic levels and formalisms are required.

Level	Meaning	Hospital example	Required?
Phonology	Sound patterns and pronunciation	Different pronunciations of a drug name	Not required for typed-text search; useful for speech input
Morphology	Internal word structure and word forms	fracture + plural -s; nephrectomy + plural -ies	Essential
Syntax	How words form phrases and sentences	"no evidence of fracture"	Useful for context and negation; not needed for first-pass indexing
Semantics	Meaning and relations	renal is related to kidney; fracture is a diagnosis	Important for safe clinical expansion
Pragmatics	Meaning in situation and speaker intention	"rule out fracture" is not a confirmed diagnosis	Important for interpretation, beyond basic normalisation
Discourse	Meaning across a document or conversation	A diagnosis introduced then referred to as "the injury"	Useful for advanced retrieval; not required for lexical matching

1.2 Which levels can be safely ignored? Phonology can be excluded from the initial typed-record system because the input is text, not audio. Syntax, pragmatics and discourse should not be claimed to be solved by a regex normaliser. They may be postponed for the first indexing release, but a clinical application must retain the source sentence and avoid presenting a negated or suspected condition as confirmed. Morphology is essential because surface forms vary systematically. Semantics is needed for carefully curated synonym and related-concept expansion.

TASK 1 – MODEL FAMILIES AND AMBIGUITY

Model family	Best hospital task	Why it fits	Why alternatives are weaker
State machines	Validate patient IDs, dates and structured formats	A finite set of states can accept or reject a regular language predictably	A probabilistic model is unnecessary for fixed formats; a loose rule list is harder to audit
Formal rule systems	Apply case folding, date canonicalisation and affix spelling rules	Rules are explicit, deterministic and explainable	A state machine alone does not express all rewrite actions conveniently; probability is not needed
Probabilistic models	Rank OCR correction candidates or infer a likely language	Uncertain inputs require scores and context	A hard rule may reject plausible noisy strings; a DFA cannot rank alternatives

1.3 Ambiguity. Lexical ambiguity occurs when one surface string has different meanings or categories. “lead” may be a verb or a metal-related noun; in a clinical corpus, “culture” may refer to a laboratory test or a general process. Morphological ambiguity occurs when one form has multiple analyses, such as “findings” as a plural noun and “findings” in a fixed report heading. Segmentation ambiguity occurs when punctuation or spacing changes token boundaries: “5mg”, “5 mg” and “5-mg” may represent the same quantity, while “q.d.” may be one abbreviation rather than several tokens. Abbreviation ambiguity occurs when “MS” may mean multiple sclerosis, morphine sulfate or a different local term depending on context.

1.4 Search in this unit. Automata recognition explores state transitions. A DFA follows one state per input symbol and uses O(n) time for a string of length n with O(1) working state beyond the transition table. An NFA may have several live states; simulation uses a set of states and is typically O(n|Q|) in a straightforward implementation. Backtracking regex engines may revisit alternatives and can become very slow on badly designed nested patterns. Lexicon lookup consumes memory proportional to stored stems

and features. Annotation consumes clinician or linguist time, while rule construction consumes engineering and domain-expert effort.

TASK 1 – PROPOSED PIPELINE

1.5 Labelled pipeline. The following sequence preserves raw input and attaches evidence at every stage:

Stage	Input → Output	Formalism / operation
1. Ingestion	PDF/OCR/text → raw record	File parser and record metadata
2. Character cleanup	raw text → cleaned text plus noise flags	Unicode normalisation and conservative regex substitutions
3. Entity recognition	cleaned text → spans and entity labels	12 regular-expression patterns and DFA validators
4. Tokenisation	text → ordered tokens with offsets	Regex tokenizer using longest-match priority
5. Surface normalisation	tokens → canonical token candidates	Case folding, date/unit rules, abbreviation dictionary
6. Morphological analysis	surface form → lemma + features	Lexicon FST and orthographic rewrite transducers
7. Fallback	unknown English token → candidate stem	Porter stemmer; low-confidence flag
8. Indexing	analyses → searchable index	Inverted index with original text and metadata
9. Retrieval	clinician query → ranked records	Exact, lemma, curated synonym and fuzzy matching
10. Audit	result → trace and quality measures	Logs, relevance judgements and clinician feedback

Input: de-identified records and clinician queries. Output: an auditable index that returns record identifiers, matching spans, normalised terms and confidence/analysis labels. The original record text is never overwritten. Every transformation stores its rule identifier so that a reviewer can reconstruct why a token was normalised.

1.6 Design assumptions. The first release assumes mostly typed English text, a curated hospital abbreviation list and stable local identifier formats. The Day-4 challenge invalidates these assumptions. The revised architecture is described in Task 12.

1.7 Recommendation. Use deterministic rules for predictable formats, finite-state morphology for known word structure and probabilistic ranking only where input uncertainty exists. Do not use a single statistical model for every stage: exact structured fields benefit from transparent validation, while OCR correction benefits from ranked alternatives.

TASK 2 – BASIC REGULAR EXPRESSION PATTERNS

Operators, definitions and clinical examples

2.1 Regular expressions describe regular languages using a small set of operators. Concatenation places expressions in sequence; disjunction (|) selects alternatives; Kleene star (*) permits zero or more repetitions; plus (+) requires one or more; optionality (?) permits zero or one; character classes [] select one character from a set; ranges such as [0-9] select a range; negated classes [^...] exclude characters; anchors ^ and $ assert the start and end of a string; counters {m,n} specify repetition counts.

Operator	**Example**	**Meaning**
Concatenation	BP[ ]?[0-9]{2,3}/[0-9]{2,3}	BP followed by systolic/diastolic values
Disjunction	mg|mcg|g	One of three units
Kleene star	[0-9]*	Zero or more digits; use carefully
Plus	[0-9]+	At least one digit
Optionality	-?[0-9]+	Optional minus sign
Character class	[A-Z]	One uppercase letter
Range	[0-9]{4}	Four digits
Negated class	[^\s]+	One or more non-whitespace characters
Anchors	^[A-Z]{2}[0-9]{6}$	Entire string must match
Counter	[0-9]{1,3}	One to three digits

2.2 Matching policy. Use full-string validation for identifiers and field validators. Use finditer-style span extraction for entities embedded in narrative text. Add word boundaries where they are appropriate, but do not assume that \b handles every Unicode or punctuation case. Validate numeric ranges after regex matching: a date-shaped string such as 2026-19-42 is syntactically shaped like a date but is not a valid calendar date.

TASK 2 – TWELVE CLINICAL ENTITY PATTERNS

The patterns below are starting specifications for a synthetic English-language hospital corpus. They are intentionally conservative. The hospital must confirm its real identifier formats, approved drug vocabulary, ward conventions and local abbreviations before deployment.

Entity	Python regex	Must match / must not match
Patient identifier	\\b(?:PID[-]?)?[A-Z]{2}[0-9]{6}\\b	AB123456 or PID-AB123456; not AB12345 or AB1234567
Admission date	\\b(?:0?[1-9]|[12][0-9]|3[01])[-/](?:0?[1-9]|1[0-2])[-/](?:19|20)[0-9]{2}\\b	DD-MM-YYYY or DD/MM/YYYY; calendar validation required; not 42/13/2025
Discharge date	\\b(?:19|20)[0-9]{2}-(?:0[1-9]|1[0-2])-(?:0[1-9]|[12][0-9]|3[01])\\b	YYYY-MM-DD; not month 00 or 13
Drug + strength	\\b[A-Za-z][A-Za-z0-9-]{1,30}\\s+[0-9]+(?:\\.[0-9]+)?\\s?(?:mg|mcg|g|mL|IU)\\b	Paracetamol 500mg; not a bare number without unit
Dosage frequency	\\b(?:OD|BD|TDS|QID|BID|TID|q\\.d\\.|q\\.\\s*[0-9]+\\s*h)\\b	Approved frequency forms; not arbitrary letter strings
Lab value + unit	\\b[A-Za-z][A-Za-z0-9 -]{0,20}:?\\s*[0-9]+(?:\\.[0-9]+)?\\s?(?:mg/dL|g/dL|mmol/L|IU/L|10\\^?9/L)\\b	Haemoglobin: 12.5 g/dL; not value with unknown unit

Blood pressure	\\b(?:BP\\s*)?[0-9]{2,3}/[0-9]{2,3}\\s?(?:mmHg)?\\b	120/80 mmHg; range validation is separate
Temperature	\\b(?:Temp(?:erature)?\\s*)?[0-9]{2}(?:\\.[0-9])?\\s?°?[CF]\\b	Temp 38.2 C; not 382 C
ICD code	\\b[A-TV-Z][0-9][A-Z0-9](?:\\.[A-Z0-9]{1,4})?\\b	A valid-looking ICD code; version-specific validation required
Ward and bed	\\b(?:Ward|W)\\s*[A-Z0-9-]+\\s*[,/-]?\\s*(?:Bed|B)\\s*[A-Z0-9-]+\\b	Ward A, Bed 12; not a ward label alone
Contact number	\\b(?:\\+?91[-]?)?[6-9][0-9]{9}\\b	Indian 10-digit mobile format with optional country code; not 8 digits
Age	\\b(?:Age\\s*)?(?:0|[1-9][0-9]{0,2})\\s?(?:years?|yrs?|y)\\b	Age 45 years; range check 0–120 is separate
Symptom duration	\\b(?:for\\s+)?[0-9]+\\s?(?:hours?|days?|weeks?|months?)\\b	for 3 days; not an unqualified number

The assignment requests twelve entity types, while its inventory enumerates thirteen when admission and discharge dates are counted separately. This answer includes all thirteen listed types to avoid omitting either date field.

TASK 2 – REGEX MATCHING, PRECEDENCE AND EVALUATION

2.3 Greedy and non-greedy matching. Consider: “Give 5 mg after breakfast and 10 mg at night.” The pattern `Give .* mg` is greedy: `.*` expands as far as possible while still allowing the final `mg`, so the matched span can include both dosage expressions. The pattern `Give .*? mg` is non-greedy: `.*?` expands only as far as needed to reach the first `mg`, so the first dosage is returned. For extracting each dose independently, use a bounded pattern such as `\\b[0-9]+\s?mg\b` rather than relying on a broad wildcard.

2.4 Precedence and grouping. Concatenation binds more tightly than disjunction. The expression `A|B C` denotes either `A` or `B C`; it does not denote `(A|B) C`. For a frequency pattern, `OD|BD q.d.` may match `OD`, `BD q.d.` but not the intended combinations. Correct grouping is `(?:OD|BD|TDS|q\.d\.)`. A misplaced alternative can silently accept an incomplete entity.

2.5 Error measurement. A true positive (TP) is a correctly recognised entity span; a false positive (FP) is an incorrect span; a false negative (FN) is a gold entity missed by the pattern. Precision = TP/(TP+FP); recall = TP/(TP+FN); F1 = 2PR/(P+R). The values below are an illustrative example only, not measurements from the user's hospital corpus.

Pattern group	**TP**	**FP**	**FN**	**Illustrative cause of errors**
Identifiers	98	2	3	Legacy IDs and OCR O/0 confusion
Dates	94	4	6	Ambiguous day/month order; invalid calendar dates
Drug + strength	88	8	12	Brand spelling, missing space, unknown units
Frequency	91	5	9	Local abbreviations and punctuation
Lab values	86	7	14	Unlisted units and analyte names
BP / temperature	96	3	4	Missing labels and unusual spacing
ICD / ward / bed	89	6	11	Code version and local ward naming
Contact / age / duration	97	2	3	Context ambiguity and boundary errors

2.6 Error causes. False positives often arise because a pattern recognises shape without meaning: 99/99 resembles a blood pressure but may be a code. False negatives arise from unlisted abbreviations, spaces,

punctuation, OCR substitutions and local formats. Pattern performance must be reported separately for each entity, not hidden in one aggregate score.

TASK 2 – TEST CASES AND ERROR CONTROL

Input	Expected result	Reason
PID-AB123456	Accept	Prefix, two letters and six digits
AB12345	Reject	Only five digits
31/02/2025	Reject after calendar check	Shape matches date regex, but date is impossible
Paracetamol 500mg	Accept	Drug-strength shape
Paracetamol five mg	Reject	Written number is outside numeric grammar
BP 120/80 mmHg	Accept	Blood pressure shape
BP 900/20	Reject after range check	Syntactically shaped but implausible
Temp 38.2 C	Accept	Temperature format
Ward A Bed 12	Accept	Ward and bed components
for 3 days	Accept	Numeric duration and unit

2.7 Evaluation procedure. Build a gold file with document_id, entity_type, start_offset, end_offset and canonical_value. Match predictions to gold spans using exact span and type equality. Count TP, FP and FN for each entity. If partial overlap is clinically useful, report it as a separate relaxed metric; do not mix it with exact-match precision. Review a random sample of false positives and false negatives with a clinician.

2.8 Operational safeguards. Store the raw matched string, canonical value, regex version and validation result. Do not auto-correct patient identifiers or medication strengths solely from a fuzzy match. A candidate correction should be shown for review when more than one interpretation is possible.

Task 2 conclusion: regular expressions are effective for predictable surface formats. They are not semantic validators and cannot reliably infer whether a matched drug, diagnosis or measurement is clinically appropriate.

TASK 3 – REGEX TOKENISATION AND NORMALISATION

3.1 Token definition. A token is the longest contiguous sequence that represents a word, abbreviation, number, measurement or structured clinical item. Hyphenated medical terms are retained as one token when the hyphen joins lexical components (follow-up, post-operative). Abbreviations with full stops are retained (q.d., i.v.). Decimal values are retained (7.5). A number attached to a recognised unit is retained as one measurement token (500mg), while the analyser stores numeric and unit subfields.

A practical Python tokenizer pattern is shown below. The alternatives are ordered from more specific to more general so that a date or measurement is not split into smaller pieces.

```
TOKEN_RE = re.compile(
  r"(?i)(?:\d{4}-\d{2}-\d{2})"              # ISO date
  r"|(?:\d{1,2}[/-]\d{1,2}[/-]\d{2,4})"      # local date
  r"|(?:\d+(?:\.\d+)?\s?(?:mg/dL|g/dL|mmol/L|mg|mcg|mL|IU|kg|cm))"
  r"|(?:[A-Za-z]+(?:\.[A-Za-z]+)+\.?)"       # dotted abbreviation
  r"|(?:[A-Za-z]+(?:-[A-Za-z]+)+)"           # hyphenated term
  r"|(?:\d+(?:\.\d+)?)"                      # number
  r"|(?:[A-Za-z]+)"                          # word
  r"|(?:[^\w\s])"                            # punctuation
)
```

3.2 Maximal munch. The tokenizer chooses the longest valid token at the current position. Example 1: “500mg” is one measurement token rather than “500” and “mg”. Example 2: “q.d.” is one abbreviation rather than “q”, “.”, “d”, “.”. Example 3: “post-operative” is one hyphenated token rather than “post”, “-”, “operative”. A naive whitespace split produces the whole phrase “BP” and “120/80” but does not classify the measurement, and it leaves punctuation attached to words.

TASK 3 – NORMALISATION RULES AND EXAMPLE RECORD

Operation	Rule	Example
Case folding	Convert case for search key only	FRACTURES → fractures
Whitespace	Collapse repeated spaces	"BP 120/80" → "BP 120/80"
Date canonicalisation	Parse known date order then store ISO date	05/11/2026 → 2026-11-05 only if DD/MM is confirmed
Unit canonicalisation	Map approved aliases to standard units	"0.5 g" → "500 mg" after dimensional conversion
Abbreviation expansion	Use context-approved dictionary	"HTN" → "hypertension"
Punctuation	Remove only non-semantic punctuation from index key	Trailing comma removed; decimal point retained

3.3 Thirty clinical abbreviations for a controlled dictionary. The following are illustrative and must be approved locally because abbreviations can be ambiguous: BP=blood pressure; HR=heart rate; RR=respiratory rate; Temp=temperature; SpO2=oxygen saturation; ECG=electrocardiogram; CBC=complete blood count; Hb=haemoglobin; WBC=white blood cell count; RBC=red blood cell count; CT=computed tomography; MRI=magnetic resonance imaging; US=ultrasound; IV=intravenous; IM=intramuscular; PO=oral; OD=once daily (or right eye in ophthalmology); BD=twice daily; TDS=three times daily; QID=four times daily; PRN=as needed; NPO=nothing by mouth; ICU=intensive care unit; OPD=outpatient department; DM=diabetes mellitus; HTN=hypertension; COPD=chronic obstructive pulmonary disease; UTI=urinary tract infection; CKD=chronic kidney disease; CVA=cerebrovascular accident.

3.4 Full synthetic record before normalisation: "DISCHARGE NOTE: Patient AB123456 admitted on 05/11/2026. Dx: fractures of left wrist. BP 120/80 mmHg; Temp 38.2 C. Paracetamol 500mg PO BD for 3 days. Hb: 12.5 g/dL. Follow-up in 2 weeks."

After normalisation for indexing: "discharge note: patient ab123456 admitted on 2026-11-05. diagnosis: fracture of left wrist. blood pressure 120/80 mmHg; temperature 38.2 °C. paracetamol 500 mg oral twice daily for 3 days. haemoglobin 12.5 g/dL. follow-up in 2 weeks." The original is retained alongside this view. The date conversion assumes the hospital has confirmed DD/MM/YYYY; if not, the date remains unresolved.

TASK 3 – INFORMATION LOSS AND LIMITS

3.5 Irreversible decisions. (1) Lowercasing destroys original capitalisation, which may distinguish a name or acronym. Keep the original token. (2) Expanding an abbreviation destroys the fact that the author used a shortened form and may select the wrong sense. Store both abbreviation and expansion with a confidence/source field. (3) Converting units destroys the original representation and may introduce rounding. Store original quantity, unit and converted quantity. (4) Removing punctuation can erase decimal or sentence boundaries. Preserve offsets and original text.

Information loss is acceptable in a secondary search key when the raw source is retained and the transformation is reversible or auditable. It is not acceptable when it changes a patient identifier, medication dose, laterality, negation or a clinically meaningful qualifier.

3.6 A non-regular requirement. Matching arbitrarily nested balanced parentheses cannot be expressed by a classical regular expression over a finite automaton. The language of balanced parentheses requires unbounded memory of nesting depth. A finite-state machine has only finitely many states and cannot remember an arbitrary count. Some modern regex engines provide recursion or balancing groups, but those features go beyond classical regular expressions and are not portable.

3.7 Task conclusion. A regular layer can reliably recognise many local token shapes and perform controlled substitutions. It cannot resolve semantic ambiguity, nested structure, clinical intent or unknown language morphology. Those limitations motivate the finite-state and review components that follow.

TASK 4 – FINITE-STATE AUTOMATA

4.1 Formal definition. A deterministic finite automaton is the five-tuple $M=(Q, \Sigma, \delta, q_0, F)$, where Q is a finite set of states, Σ is the input alphabet, $\delta: Q \times \Sigma \rightarrow Q$ is the transition function, q_0 is the start state and $F \subseteq Q$ is the set of accepting states. A string is accepted when the machine ends in an accepting state after consuming every input symbol.

For readable clinical validators, symbols are grouped into classes such as LETTER, DIGIT, HYPHEN and OTHER. A transition table with class labels represents all individual characters in that class. Every unlisted transition goes to a dead state.

Automaton	Language	State outline
A1 Patient ID	Two uppercase letters followed by six digits	q0 → LETTER → q1 → LETTER → q2 → DIGIT×6 → q8 accept
A2 ISO date	YYYY-MM-DD with numeric month/day positions	q0…q10; separators required at fixed positions
A3 BP	2–3 digits / 2–3 digits, optional mmHg	start → systolic digits → slash → diastolic digits → optional unit → accept
A4 Mobile	Optional +91 and ten digits beginning 6–9	start → country prefix optional → valid first digit → nine digits → accept

4.2 DFA A1 patient identifier. States q0 start, q1 after first letter, q2 after second letter, q3–q8 after one to six digits, qD dead. q8 is the only accepting state. Transitions: q0 on LETTER→q1; q1 on LETTER→q2; q2 on DIGIT→q3; q3→q4; q4→q5; q5→q6; q6→q7; q7→q8. Any other symbol goes to qD; q8 on any additional symbol goes to qD.

State	LETTER	DIGIT	OTHER	Accept?
q0	q1	qD	qD	No
q1	q2	qD	qD	No
q2	qD	q3	qD	No
q3	qD	q4	qD	No

q4	qD	q5	qD	No
q5	qD	q6	qD	No
q6	qD	q7	qD	No
q7	qD	q8	qD	No
q8	qD	qD	qD	Yes
qD	qD	qD	qD	No

TASK 4 – DFA VALIDATORS AND NFA

4.3 DFA A2 ISO date. The input shape is exactly ten characters: four digits, hyphen, two digits, hyphen, two digits. A position-based DFA uses q0–q10 and a dead state. q0–q3 accept digits in year positions; q4 accepts hyphen; q5–q6 accept digits in month positions; q7 accepts hyphen; q8–q9 accept digits in day positions; q10 accepts only at end. A separate calendar validator checks month lengths and leap years.

4.4 DFA A3 blood pressure. Accept two or three systolic digits, slash, two or three diastolic digits, and optional whitespace plus mmHg. For clarity, use a DFA for the numeric skeleton and a deterministic post-check for physiological ranges. This avoids encoding every numeric comparison in a large state machine.

4.5 DFA A4 mobile number. Accept either ten digits beginning 6–9 or the prefix +91 followed by ten digits beginning 6–9. The optional prefix creates branching in an NFA design, but a DFA can still be built by tracking the prefix path and digit count.

4.6 NFA with epsilon transitions. Consider an identifier with optional literal prefix `PID-`: either `AB123456` or `PID-AB123456`. The NFA starts at s0 and has an epsilon transition to two branches: a no-prefix branch expecting two letters and six digits, and a prefix branch expecting P-I-D-hyphen followed by the same letter/digit sequence. This is convenient because the optional prefix is represented as an alternative path.

Input position	**Live NFA states (illustrative)**	**Explanation**
Before input	{s0, s1, p0}	ε-closure activates both alternatives
A	{s2}	No-prefix branch consumes first letter
B	{s3}	Consumes second letter

1	{s4}	First digit
2…6	{s5…s9}	Remaining digits advance one state each
End	{s9}	Accepting state reached after six digits

Accepted trace: AB123456 follows the no-prefix branch and reaches its accepting state. Rejected trace: AB12345 stops one digit early; the live state set contains a non-final digit state only, so the string is rejected. The prefixed path accepts PID-AB123456.

TASK 4 – SUBSET CONSTRUCTION AND MINIMISATION

4.7 Subset construction. An NFA is converted into a DFA by treating each DFA state as a set of NFA states. The start state is ε-closure({s0}). For each input symbol a, compute ε-closure(move(S,a)). A subset is accepting if it contains at least one NFA final state. Empty subset becomes the dead state.

DFA subset	On LETTER	On DIGIT	On P/I/D/-	Accept?
{s0,s1,p0}	{s2}	∅	{p1} on P	No
{s2}	{s3}	∅	∅	No
{s3}	∅	{s4}	∅	No
{s4}	∅	{s5}	∅	No
{s5}	∅	{s6}	∅	No
{s6}	∅	{s7}	∅	No
{s7}	∅	{s8}	∅	No
{s8}	∅	{s9}	∅	No
{s9}	∅	∅	∅	Yes
{p1}	∅	∅	{p2} on I	No
{p2}	∅	∅	{p3} on D	No
{p3}	∅	∅	{p4} on hyphen	No
{p4}	{s2}	∅	∅	No

∅	∅	∅	∅	No

The table is a compact class-level illustration; a production subset-construction table must enumerate each literal prefix symbol and every reachable subset. After construction, equivalent states are merged if no future suffix distinguishes them. The common suffix validator after the two-letter prefix can be shared, reducing duplicated paths. The exact minimal state count depends on whether the prefix is included in the accepted language and whether the alphabet is character-level or class-level.

4.8 Minimisation method. Partition states into accepting and non-accepting sets. Repeatedly split a partition when states transition to different partitions for any input class. Stop when no partition changes. For the example, the two prefix routes converge at the state expecting the first of two letters, so their common suffix states can be merged. A state-count report should be generated from the actual transition graph rather than guessed.

TASK 4 – THOMPSON, EQUIVALENCE AND PUMPING LEMMA

4.9 Thompson construction. For the expression `[A-Z]{2}[0-9]{6}`, build a two-state fragment for each character class and concatenate the fragments using epsilon links. A repetition counter is expanded into the required number of fragments. The resulting NFA accepts exactly two uppercase letters followed by six digits. Conversely, state elimination can derive a regular expression from a finite automaton: add a new start and final state, then eliminate intermediate states while updating edge labels with union, concatenation and Kleene star.

4.10 Equivalence. Thompson's construction preserves the language: every path through the constructed NFA corresponds to a sequence permitted by the regex, and every permitted regex string has a corresponding accepting path. State elimination preserves the language in reverse. Equivalence should be checked by testing representative accepted/rejected strings and, preferably, by constructing a symmetric-difference automaton and checking that no accepting state is reachable.

4.11 Pumping lemma. If L is regular, there is a pumping length p such that every string $s \in L$ with $|s| \geq p$ can be written $s=xyz$, with $|xy| \leq p$, $|y|>0$, and $xy^iz \in L$ for every $i \geq 0$.

Consider $L=\{ '('^n ')'^n : n \geq 0 \}$, the language of strings with equal numbers of opening and closing parentheses in the required order. Assume L is regular and let p be its pumping length. Choose s='('*p + ')'^p. Since $|xy| \leq p$, the non-empty substring y consists only of opening parentheses. Pump down with i=0. The resulting string has fewer opening parentheses than closing parentheses, so it is not in L. This contradicts

the pumping lemma; therefore L is not regular. This proves why classical finite-state regex cannot validate arbitrary nested balanced parentheses.

TASK 5 – INFLECTIONAL MORPHOLOGY

5.1 Definitions. A morpheme is the smallest unit that contributes meaning or grammatical function. A stem is the form to which an inflectional affix attaches. An affix is a bound morpheme attached before or after a stem. Inflection creates grammatical variants without normally changing the lexical category or core dictionary meaning. A paradigm is the set of related inflected forms of a lexeme.

Inflection generally preserves category: fracture (noun) → fractures (plural noun); treat (verb) → treated (past-tense verb). Derivation can create a new lexeme or category: infect (verb) → infection (noun). Inflection is usually more productive and occurs outside derivational material in many English word forms, although real morphology has exceptions.

Paradigm	**Forms**	**Clinical lexeme example**
Noun number	singular / plural	fracture / fractures
Verb tense	base / past / participle	treat / treated / treated
Progressive aspect	base / -ing	bleed / bleeding
Third-person agreement	base / -s	the patient reports / nurse reports
Adjective comparison	positive / comparative / superlative	high / higher / highest

TASK 5 – PARADIGMS AND SPELLING RULES

Lexeme	Inflectional forms	Features
fracture	fracture, fractures	N; SG, PL
wound	wound, wounds	N; SG, PL
treat	treat, treats, treated, treating	V; base, 3SG, past/participle, progressive
bleed	bleed, bleeds, bled, bleeding	V; irregular past
high	high, higher, highest	ADJ; positive, comparative, superlative

5.2 English orthographic rules. E-deletion: drop final silent e before a vowel-initial suffix (treat + ing → treating; make + ing → making). Counter-example: dye + ing → dyeing, where deleting e would produce "dying" and change the meaning. Consonant doubling: double a final consonant after a stressed short vowel in suitable monosyllabic forms (stop + ed → stopped). Counter-example: visit + ed → visited, not "visitted". Y-to-i: consonant+y changes to i before many suffixes (injury + s → injuries; carry + ed → carried). Counter-example: play + ed → played because a vowel precedes y; before -ing, y is retained (carrying). Sibilant -es: add -es after s, x, z, ch or sh (diagnosis is irregular; watch → watches). Counter-example: quiz + es requires spelling adjustment to quizzes; a simplistic append-only rule gives "quizes".

5.3 Irregular forms. A hospital lexicon should list common irregulars such as diagnosis/diagnoses, analysis/analyses, crisis/crises, child/children, foot/feet, tooth/teeth, man/men, woman/women, mouse/mice, be/was/were/been, go/went/gone, have/has/had, do/does/did, bleed/bled, fall/fell/fallen and take/took/taken. These cannot be generated reliably by the regular suffix rules because the stem changes internally or the form is suppletive.

TASK 5 – MEASUREMENT AND RETRIEVAL EFFECT

5.4 Type-to-lemma ratio. Let V be the number of distinct surface word types and L the number of distinct lemmas after morphological normalisation. Type-to-lemma ratio = V/L. A higher ratio indicates more surface variation per lemma. For example, if a synthetic sample contains 1,800 distinct word forms and 1,200 lemmas, the ratio is 1.50. This is an illustrative calculation, not a measured corpus result.

5.5 Inflection-only query failures. Suppose 40 clinician queries are judged against the corpus. If 12 queries fail under exact surface matching and 8 of those become successful after plural/tense normalisation, the proportion of exact-match failures explained by inflection is 8/12 = 66.7%. Report the numerator and denominator, and distinguish "query-level success" from "document-level recall".

Metric	Formula	Illustrative value
Surface types	Count unique token strings	1,800
Lemma count	Count unique canonical lemmas	1,200
Type-to-lemma ratio	V/L	1.50
Exact failures	Queries with no relevant result	12 of 40
Inflection-resolved failures	Failures recovered by inflection only	8 of 12 = 66.7%

5.6 Interpretation. Inflectional analysis is a high-value normalisation step because it connects predictable grammatical variants while retaining the same lexical identity. It should be evaluated on hand-annotated forms and clinician queries, not inferred from a few examples. Irregular forms need lexicon entries and should be tested separately.

TASK 6 – DERIVATIONAL MORPHOLOGY

6.1 Definition. Derivation creates a new lexeme by attaching an affix to a base. It may change word class, add a specialised meaning or produce a clinical term whose meaning is only partly predictable from its components. Derivation is harder than inflection because it is less uniformly productive, may change category, may be lexically restricted and may involve semantic specialisation.

Affix	Type	Example	Typical effect
-itis	Suffix	arthritis	Noun: inflammation
-ectomy	Suffix	nephrectomy	Noun: surgical removal
-ology	Suffix	cardiology	Noun: field of study
-osis	Suffix	fibrosis	Noun: condition/process
-aemia	Suffix	anaemia	Noun: blood condition
-ation	Suffix	amputation	Verb → noun
-ment	Suffix	treatment	Verb → noun
-able	Suffix	treatable	Verb → adjective
-ive	Suffix	sedative	Verb/noun base → adjective/noun
-al	Suffix	renal	Noun base → adjective
nephro-	Prefix	nephrology	Kidney-related compound
cardio-	Prefix	cardiovascular	Heart-related compound
hyper-	Prefix	hypertension	Excess/high
hypo-	Prefix	hypoglycaemia	Low/deficient
re-	Prefix	readmission	Again/back
un-	Prefix	unresponsive	Negation/reversal
dys-	Prefix	dysfunction	Abnormal/difficult
brady-	Prefix	bradycardia	Slow

tachy-	Prefix	tachycardia	Fast
peri-	Prefix	pericardial	Around

TASK 6 – DERIVATIONAL FAMILIES AND SEMANTICS

6.2 Derivational families. The examples below show plausible lexical families; not every combination is a valid clinical word.

Root/family	Attested or plausible family	Attachment order
infect	infect → infection → infectious → reinfection	re- + infection; -ion then -ous
operate	operate → operation → operative → postoperative	post- + operative
treat	treat → treatment → treatable → untreated	-ment/-able; un- + treated
fract	fracture → fractured → fracturing	-ure lexical base; inflection follows
nephr	nephron → nephritis → nephrectomy → nephrology	Greek combining form; distinct lexical formations
cardi	cardiac → cardiology → cardiologist	-ology then -ist

Impossible or unsupported orders. `-itis + nephro-` cannot be attached as if a suffix were a free prefix; the combining form nephro- precedes the base. `un- + -ation + treat` is not a valid linear order for "untreatment"; the normal lexical forms are untreated or treatment. Affix order is constrained by category and lexical selection.

6.3 Non-compositionality. "Inflammation" is not simply any action of inflaming; it is a clinical condition. "Nephrectomy" means surgical removal of a kidney, not any action involving nephro- and -ectomy in an unrestricted sense. "Culture" in a medical note may refer to a laboratory procedure or the resulting specimen/test. "Discharge" may refer to release from hospital or fluid from a body site. Aggressive stemming may conflate unrelated senses or reduce meaningful terms to fragments.

6.4 Search implication. Derivational relatives should be linked using curated concept relations and context, not always collapsed into one stem. A query for “renal” may reasonably retrieve “nephrology” or “nephrectomy” in a related-concept expansion, but these should be labelled as related concepts rather than exact morphological matches.

TASK 6 – PRODUCTIVITY AND CONFLATION

6.5 Productivity measurement. For each affix, count distinct derived word types and total token occurrences. An affix is productive if it attaches to multiple bases and appears in new or varied formations; it is fossilised or restricted if it occurs in a small set of fixed lexical items. The counts below are a sample reporting format only. Replace with counts from the actual corpus.

Affix	Illustrative types	Illustrative tokens	Classification / reason
-ation	18	146	Productive; many verb-to-noun forms
-able	12	52	Productive; several adjective formations
-al	24	220	Productive but lexically constrained
-itis	9	41	Specialised; limited medical bases
-ectomy	7	18	Restricted clinical formation
-ology	11	34	Moderately productive in specialty names
-osis	14	73	Specialised condition nouns
hyper-	8	31	Productive clinical prefix
hypo-	10	44	Productive clinical prefix

re-	15	67	Productive general prefix
-aemia	5	12	Restricted/fossilised medical suffix
-ive	13	49	Productive but category-sensitive

6.6 Conflation helps. (1) infection/infections: plural variation should be connected. (2) readmission/readmissions: plural and related admission retrieval can help locate repeat-care records, but should preserve the "re-" meaning. (3) fracture/fractures: noun number should be conflated. (4) treated/treating: tense/aspect variation should be connected.

6.7 Conflation misleads. (1) hypertension and hypotension share the base "tension" but mean opposite blood-pressure states; removing prefixes is unsafe. (2) nephrectomy and nephritis share nephro-related material but describe different clinical events/conditions. (3) infectious and infection are related but may have different syntactic roles and clinical implications. (4) unresponsive and responsive have opposite meanings; stripping un- destroys polarity.

6.8 Decision. Inflectional relatives can normally be conflated in the main lemma index. Derivational relatives should remain separate entries connected through typed relations such as DERIVED_FROM, RELATED_CONCEPT or ANTONYM. Negation, laterality, dosage and polarity must never be discarded.

TASK 7 – FINITE-STATE MORPHOLOGICAL PARSING

7.1 Finite-state transducer. An FST is a finite-state machine whose transitions read symbols on an input tape and emit symbols on an output tape. For morphology, the lexical tape contains a lemma and grammatical features; the surface tape contains the written word. A transducer can be run from lexical to surface direction to generate forms, or from surface to lexical direction to analyse forms.

Worked example: lexical input `fracture+N+PL` is mapped to surface `fractures`. The transducer copies the stem `fracture`, realises the plural feature as `s`, and outputs the resulting surface form. In the reverse direction, the same relation consumes `fractures` and emits `fracture+N+PL`.

7.2 Continuation classes. A compact lexicon groups stems by part of speech and permitted endings. Noun class N_REG permits plural -s/-es; N_Y permits y-to-ies; N_IRREG points to listed irregular forms. Verb

class V_REG permits -s, -ed and -ing; V_IRREG lists forms such as bleed/bled. Adjective class ADJ permits -er/-est where appropriate. Clinical derivational classes restrict affixes to valid bases.

Continuation class	Lexicon examples	Permitted features
N_REG	fracture, wound, infection	N, SG; N, PL
N_Y	injury, artery	N, SG; N, PL with y→ies
N_IRREG	diagnosis, analysis	N, SG; N, PL from lexicon
V_REG	treat, monitor, admit	V, base; 3SG; past; progressive
V_IRREG	bleed, fall, take	Irregular past/participle listed
ADJ	high, low, severe	Positive; comparative; superlative where valid

TASK 7 – HAND TRACES

Surface form	Input consumed	Output emitted	Analysis
fractures	fracture + s	fracture + N + PL	fracture+N+PL
wounds	wound + s	wound + N + PL	wound+N+PL
treated	treat + ed	treat + V + PAST	treat+V+PAST
bleeding	bleed + ing	bleed + V + PROG	bleed+V+PROG
diagnoses	diagnos- + es / lexicon path	diagnosis + N + PL	diagnosis+N+PL

7.3 State sequence for fractures. q0 reads lexical stem `fracture` and moves to noun continuation state qN. qN accepts singular with no suffix or reads plural feature +PL. The plural realisation emits `s` and reaches qFinal. On the surface-to-lexical path, the transducer reads the letters f-r-a-c-t-u-r-e-s; the stem portion is aligned to `fracture`, and final s is analysed as +PL. The exact symbol-by-symbol alignment can be represented with identity arcs for stem letters and an affix arc for the final s.

7.4 Ambiguity. `wound` may be a noun (an injury) or the past tense of the verb wind in general English. The clinical lexicon strongly favours the noun in a phrase such as “wound dressing”, but context is required. `left` may be a direction adjective or the past tense of leave. The transducer should return all

lexicon-supported analyses; a downstream contextual ranker uses neighbouring words and section type to rank them.

TASK 7 – GENERATION, AMBIGUITY AND IMPLEMENTATION

7.5 Generation with the same machine. Lexical input `fracture+N+PL` follows the lexical-to-surface direction and produces `fractures`. Input `treat+V+PAST` produces `treated`; input `bleed+V+PAST` produces `bled` through an irregular lexical entry. Parsing is the reverse relation, not a separately invented rule set. This bidirectionality is a core advantage of an FST.

7.6 Ambiguity handling. Return every valid analysis with a score or rank. Do not discard ambiguity at the morphology stage. The next component can use local syntax, document section, a clinical terminology map or clinician feedback. If two analyses remain equally plausible, retain both in the index.

7.7 Lexicon size. The assignment asks for at least 300 stems. Build a UTF-8 CSV with lemma, part_of_speech, continuation_class, clinical_domain, source and review_status. The first 300 entries should be drawn from approved clinical vocabulary, common verbs, measurement terms and record headings. A list generated from a public dictionary is not a substitute for clinician validation.

7.8 FST limitations. A finite-state transducer handles local morphology and spelling relations well, but it does not understand diagnosis certainty, negation scope, clinical causality or long-distance discourse. It also cannot analyse a word absent from the lexicon unless a productive rule safely covers it.

TASK 8 – ORTHOGRAPHIC RULES AND TWO-LEVEL MORPHOLOGY

8.1 Why a lexicon FST is not enough. Simply concatenating a stem and suffix often gives an incorrect spelling. `injury + s` should become `injuries`, not `injurys`. `treat + ing` becomes `treating` after deleting final e. `stop + ed` becomes `stopped` through consonant doubling. `diagnosis + plural` becomes `diagnoses`, an irregular alternation. Orthographic transducers handle these changes without storing every surface form as a separate stem.

Underlying sequence	Naive output	Correct surface	Rule
injury + s	injurys	injuries	y→i before plural s, with es realisation
treat + ing	treating (acceptable after e deletion)	treating	Delete final e before vowel suffix
stop + ed	stoped	stopped	Double final consonant after short stressed vowel
watch + s	watchs	watches	Add e before sibilant plural
diagnosis + s	diagnosiss	diagnoses	Lexical irregular alternation

TASK 8 – REWRITE RULES AND CASCADE

8.2 Rewrite rules. The notation a → b / c __ d means rewrite a as b when preceded by c and followed by d. The following rules are simplified English orthographic rules; lexical exceptions are handled separately.

- Final e deletion: e → 0 / V __ +V (delete final e before a vowel-initial suffix), except protected lexical items such as dye + ing.
- Consonant doubling: C → CC / V̆ __ +V where the base ends in a single consonant after a short stressed vowel and suffix begins with a vowel; lexical and stress exceptions apply.
- Y-to-i: y → i / C __ +{s, ed, er, est}, except before -ing and after a vowel.
- Sibilant plural: +s → es / {s, x, z, ch, sh} __ #.
- Plural y realisation: +s → ies and final y → 0 / C y __ +PL.

8.3 Cascade. Lexicon FST → morphological feature realisation → orthographic rule transducers → surface string. The order matters: select the morpheme boundary and suffix first; apply y-to-i or e-deletion at the boundary; apply doubling when the stem and suffix context is known; then perform final surface formatting. Irregular lexical entries bypass the productive rules.

8.4 Sequential cascade versus two-level morphology. A sequential cascade applies one transducer after another. It is easy to debug because each stage has a clear input and output, but rule order can affect results. Koskenniemi-style two-level morphology relates lexical and surface symbols in parallel using constraints at corresponding positions. It can express several spelling alternations simultaneously and avoids some cascade ordering problems, but the rule system and debugging can be harder for beginners.

TASK 8 – TESTS AND FAILURE ACCOUNTING

Test input	Expected surface	Expected analysis	Status in rule model
fracture+N+PL	fractures	fracture+N+PL	Pass
wound+N+PL	wounds	wound+N+PL	Pass
injury+N+PL	injuries	injury+N+PL	Pass
artery+N+PL	arteries	artery+N+PL	Pass
treat+V+PROG	treating	treat+V+PROG	Pass
make+V+PROG	making	make+V+PROG	Pass
stop+V+PAST	stopped	stop+V+PAST	Pass
visit+V+PAST	visited	visit+V+PAST	Pass
watch+N+PL	watches	watch+N+PL	Pass
box+N+PL	boxes	box+N+PL	Pass
diagnosis+N+PL	diagnoses	diagnosis+N+PL	Lexicon exception
analysis+N+PL	analyses	analysis+N+PL	Lexicon exception
dye+V+PROG	dyeing	dye+V+PROG	Protected exception
play+V+PAST	played	play+V+PAST	Pass
carry+V+PROG	carrying	carry+V+PROG	Pass
bleed+V+PAST	bled	bleed+V+PAST	Lexicon exception
admit+V+PAST	admitted	admit+V+PAST	Pass if stress rule licensed
prefer+V+PAST	preferred	prefer+V+PAST	Pass if stress rule licensed
fix+N+PL	fixes	fix+N+PL	Pass
church+N+PL	churches	church+N+PL	Pass

8.5 Failure accounting. Rule failures should be assigned to one of four causes: missing lexicon entry, irregular form not listed, overgeneralised spelling rule or incorrect morpheme boundary. A rule that produces a plausible English word is not automatically correct for a clinical term. The test table includes exceptions deliberately so that the system is not evaluated only on easy regular forms.

8.6 Conclusion. The lexicon FST provides lexical structure; orthographic transducers provide spelling realisation. A cascade is a practical first implementation, while a two-level system is appropriate when the lexicon and alternation inventory become large.

TASK 9 – THE PORTER STEMMER

9.1 Purpose. Porter stemming is a suffix-stripping algorithm designed to reduce related English word forms to a common stem without a dictionary. It is not a lemmatiser: the output may not be a valid word. This speed and simplicity are useful for broad information retrieval, but the algorithm may over-stem or under-stem specialised clinical vocabulary.

9.2 Consonants and vowels. In the Porter algorithm, a, e, i, o, u are vowels. y is treated contextually: it is a consonant at the beginning of a word and otherwise has the opposite status of the preceding letter. A word can be represented as sequences of consonant and vowel groups. The measure m counts the number of VC sequences in the stem.

The condition *v means the stem contains a vowel. The condition *d means the stem ends in a double consonant. The condition *o means the stem ends in consonant-vowel-consonant where the final consonant is not w, x or y. These conditions control suffix removal and e insertion.

9.3 Worked measure examples. `fracture`: approximate C/V groups are fr (C), a (V), ct (C), u (V), r (C), e (V); m is the number of VC transitions, with final silent-e treatment depending on the step. `treated`: treat + ed; after removing -ed the stem `treat` contains a vowel, satisfying *v. `bleeding`: removal of -ing leaves `bleed`; it has a double final consonant, satisfying *d. `wounds`: removal of -s leaves `wound`. `infection`: suffix -ion is considered only under its step-specific conditions. `hypertension`: long clinical compounds may be reduced in ways that are not linguistically ideal.

TASK 9 – FIVE STEPS AND HAND TRACES

9.4 Porter algorithm overview. Step 1 handles plurals and -ed/-ing forms; Step 2 maps longer derivational suffixes such as -ational; Step 3 removes selected suffixes such as -icate/-ative; Step 4 removes suffixes such as -ement/-ment/-ion when m is greater than one and other conditions hold; Step 5 removes final -e and reduces -ll under measure conditions. Exact suffix ordering and conditions must follow the selected Porter implementation.

Word	Likely operation	Illustrative stem	Clinical caution
fractures	Step 1: remove plural -s	fracture	Correctly connects plural and singular
treated	Step 1: remove -ed	treat	Useful inflectional conflation
bleeding	Step 1: remove -ing	bleed	Correct only if the stem is recognised
wounds	Step 1: remove -s	wound	Useful plural handling
infections	Step 1 plural; later suffix rules may apply	infect	May produce a non-lemma stem
nephrectomies	Suffix handling may not restore lexical base	nephrectomi / variant	Clinical derivation and y/ies require lexicon support

These are explanatory traces, not a promise that every Porter implementation returns exactly the displayed intermediate form. The official algorithm has ordered suffix tables and measure conditions. The reproducible Python code in Task 10 should be used to obtain exact outputs from the installed NLTK PorterStemmer, and those outputs should be copied into the final hand-trace table.

TASK 9 – STEMMING, LEMMATISATION AND ERRORS

9.5 Stemming versus lemmatisation. Stemming removes endings using heuristic rules. Lemmatisation uses lexical and morphological knowledge to return a valid lemma with grammatical analysis. For example, Porter may reduce `fractures` to `fractur`, while a morphological analyser returns `fracture+N+PL`. For `bled`, a lexicon-based analyser can return `bleed+V+PAST`; a suffix stemmer may not recover that irregular base.

Comparison	Porter stemming	Finite-state lemmatisation
Resource	Suffix rules; no full lexicon required	Lexicon, continuation classes and rules
Output	Stem may not be a word	Lemma plus grammatical features
Speed	Usually fast	Fast after resources are compiled
Unknown words	May still strip suffixes	May return unknown or fallback
Clinical precision	Risk of conflating distinct terms	Higher when lexicon and rules are curated
Maintenance	Algorithm fixed; vocabulary less explicit	Lexicon and rules need expert maintenance

9.6 Over-stemming. Over-stemming merges unrelated words. In clinical text, stripping prefixes can merge hypertension and hypotension, which have opposite meanings. Removing derivational material may also merge nephritis and nephrectomy despite their different meanings. Under-stemming occurs when related forms remain separate, such as an irregular plural or a specialised term not handled by the suffix list.

9.7 Error rates. On 200 annotated forms, define over-stemming rate as incorrectly merged forms divided by 200; under-stemming rate as related forms not conflated divided by 200. The assignment requires actual annotation. Example only: 12 over-stem errors / 200 = 6%; 18 under-stem errors / 200 = 9%. Do not report these illustrative figures as measured results.

TASK 9 – RETRIEVAL EXPERIMENT

9.8 Retrieval protocol. Use the same 40 clinician queries and relevance judgements in three conditions: (A) exact surface matching with no normalisation; (B) Porter stemming; (C) finite-state morphological analysis. For every query, record retrieved relevant documents, all retrieved documents and all relevant documents. Calculate precision, recall and F1 per query and macro-average across queries.

Condition	Illustrative precision	Illustrative recall	Illustrative F1	Interpretation
No normalisation	0.92	0.55	0.69	High precision but misses inflectional variants
Porter stemmer	0.78	0.76	0.77	Improved recall; some clinically unsafe conflation
FST morphology	0.90	0.84	0.87	Better balance when lexicon is adequate

9.9 Recommendation from the experiment. A higher recall alone is not enough. If a stemmer retrieves hypotension for a hypertension query, the result can be harmful even if aggregate recall rises. The FST approach is preferred for known clinical vocabulary because it preserves feature distinctions and can be audited. Porter is a fallback candidate generator for unknown English words, with lower ranking and a visible “approximate stem match” label.

TASK 10 – IMPLEMENT AND TEST THE PIPELINE

10.1 Implementation environment. The brief permits the Python standard library and NLTK only. The following core program uses `re`, `datetime`, `time`, `collections` and NLTK's PorterStemmer. The finite-state components are represented with explicit dictionaries and transition functions so they remain inspectable. No external machine-learning package is required.

```
import re, time
from datetime import datetime
from collections import defaultdict
from nltk.stem import PorterStemmer

TOKEN_RE = re.compile(
    r"\d{4}-\d{2}-\d{2}|\d{1,2}[/-]\d{1,2}[/-]\d{2,4}|"
    r"\d+(?:\.\d+)?\s?(?:mg/dL|g/dL|mmol/L|mg|mcg|mL|IU|kg|cm)|"
    r"[A-Za-z]+(?:\.[A-Za-z]+)+\.?|[A-Za-z]+(?:-[A-Za-z]+)+|"
    r"\d+(?:\.\d+)?|[A-Za-z]+|[^\w\s]", re.I)

PATTERNS = {
 "patient_id": re.compile(r"\b(?:PID[- ]?)?[A-Z]{2}\d{6}\b", re.I),
 "date": re.compile(r"\b(?:\d{1,2}[/-]\d{1,2}[/-]\d{4}|\d{4}-\d{2}-\d{2})\b"),
 "drug_strength": re.compile(r"\b[A-Za-z][A-Za-z0-9-
]{1,30}\s+\d+(?:\.\d+)?\s?(?:mg|mcg|g|mL|IU)\b", re.I),
 "frequency": re.compile(r"\b(?:OD|BD|TDS|QID|BID|TID|PRN)\b", re.I),
 "blood_pressure": re.compile(r"\b(?:BP\s*)?\d{2,3}/\d{2,3}\s?(?:mmHg)?\b", re.I),
 "temperature": re.compile(r"\b(?:Temp\s*)?\d{2}(?:\.\d)?\s?°?[CF]\b", re.I),
 "age": re.compile(r"\b(?:Age\s*)?\d{1,3}\s?(?:years?|yrs?|y)\b", re.I),
 "duration": re.compile(r"\b(?:for\s+)?\d+\s?(?:hours?|days?|weeks?|months?)\b", re.I)
}

ABBR = {"bp":"blood pressure", "htn":"hypertension",
        "dm":"diabetes mellitus", "iv":"intravenous",
        "po":"oral", "prn":"as needed", "cbc":"complete blood count"}

def tokenize(text):
```

```
    return TOKEN_RE.findall(text)

def normalise(text):
    text = re.sub(r"\s+", " ", text).strip()
    return text.casefold()

def expand_abbreviations(tokens):
    return [ABBR.get(t.casefold(), t) for t in tokens]

def validate_patient_id(value):
    return bool(re.fullmatch(r"(?:PID[- ]?)?[A-Z]{2}\d{6}", value, re.I))

def validate_iso_date(value):
    try:
        datetime.strptime(value, "%Y-%m-%d")
        return True
    except ValueError:
        return False

def entity_matches(text):
    return {name: [(m.group(), m.start(), m.end())
                   for m in pattern.finditer(text)]
            for name, pattern in PATTERNS.items()}

# A small explicit DFA for two letters followed by six digits.
def patient_dfa(s):
    state = 0
    for ch in s:
        if state in (0, 1) and ch.isalpha() and ch.isupper():
            state += 1
        elif 2 <= state <= 7 and ch.isdigit():
            state += 1
        else:
```

```
            return False
    return state == 8

# Lexical-to-surface and surface-to-lexical relation.
LEXICON = {
 "fracture": ("N", "N_REG"),
 "wound": ("N", "N_REG"),
 "treat": ("V", "V_REG"),
 "bleed": ("V", "V_IRREG")
}
IRREGULAR = {"bled": ("bleed", "V", "PAST")}

def analyse(word):
    w = word.casefold()
    if w in IRREGULAR:
        lemma, pos, feat = IRREGULAR[w]
        return [(lemma, pos, feat)]
    if w in LEXICON:
        return [(w, LEXICON[w][0], "BASE")]
    if w.endswith("s") and w[:-1] in LEXICON:
        return [(w[:-1], LEXICON[w[:-1]][0], "PL")]
    if w.endswith("ed") and w[:-2] in LEXICON:
        return [(w[:-2], LEXICON[w[:-2]][0], "PAST")]
    if w.endswith("ing") and w[:-3] in LEXICON:
        return [(w[:-3], LEXICON[w[:-3]][0], "PROG")]
    return []

def generate(lemma, pos, feature):
    if feature == "BASE":
        return lemma
    if feature == "PL":
        return lemma + "s"
    if feature == "PAST":
```

```
        if lemma == "bleed": return "bled"
        return lemma + "ed"
    if feature == "PROG":
        return lemma[:-1] + "ing" if lemma.endswith("e") else lemma + "ing"
    return None

porter = PorterStemmer()
def run_record(text):
    t0 = time.perf_counter()
    toks = tokenize(text)
    clean = normalise(text)
    entities = entity_matches(text)
    analyses = {t: analyse(t) for t in toks if t.isalpha()}
    stems = {t: porter.stem(t) for t in toks if t.isalpha()}
    elapsed = time.perf_counter() - t0
    return toks, clean, entities, analyses, stems, elapsed

if __name__ == "__main__":
    sample = ("Patient AB123456 admitted 2026-11-05. "
              "Fractures noted. Paracetamol 500mg PO BD for 3 days. "
              "BP 120/80 mmHg; Temp 38.2 C.")
    result = run_record(sample)
    print("TOKENS:", result[0])
    print("ENTITIES:", result[2])
    print("ANALYSES:", result[3])
    print("PORTER:", result[4])
    print("RUNTIME_SECONDS:", round(result[5], 6))
```

TASK 10 – TESTING AND REPRODUCIBILITY

10.2 Installation. Install Python, then install NLTK with `python -m pip install nltk`. The code uses only NLTK's PorterStemmer and does not require downloading a corpus. Save the program as `hospital_normaliser.py` and run it from a terminal using `python hospital_normaliser.py`. If the machine uses the Windows Python launcher, `py hospital_normaliser.py` is also valid.

10.3 Test categories. Test every automaton with at least one accepted and one rejected string. Test every regex with a positive and negative case. Include ambiguous forms from Task 7, irregulars from Task 5, date boundary cases, decimal measurements, and OCR-like substitutions. Compare program output with the manually derived results. Record Python version, NLTK version, date, hardware and corpus version.

Component	Positive test	Negative test	Expected check
Patient DFA	AB123456	AB12345	Accept exactly 8 symbols
ISO date	2026-11-05	2026-19-45	Calendar-valid date only
BP pattern	120/80 mmHg	120-80	Slash and digits required
Morphology	fractures	fracturings	Known analysis only
Irregular	bled	bleeded	Return bleed+V+PAST
Tokenizer	500mg		Measurement is one token
Abbreviation	HTN	unknown acronym	Expand only dictionary entries

10.4 Stage metrics. For each stage report number of input items, output items, rejected items and runtime. Example: 25,000 tokens enter tokenisation; if 24,700 are emitted as tokens and 300 are empty/filtered fragments, report those values. For entity recognition, count spans by type. For morphology, count analysed tokens, ambiguous tokens and unknown tokens. Run time should be measured using time.perf_counter() and averaged over repeated runs.

TASK 10 – RESULTS TABLES AND PLOT

10.5 Illustrative pipeline results. The table below demonstrates the required reporting structure. It is not a claim that the supplied PDF contained a dataset or that these values were measured.

Stage	Items in	Processed/matched	Rejected/unknown	Illustrative runtime
Ingestion	500 records	500	0	0.8 s
Tokenisation	25,000 words approx.	25,000 tokens	0	0.2 s
Entity patterns	25,000 tokens	1,420 spans	65 candidate rejects	0.15 s
Morphology	12,000 word tokens	9,100 analyses	2,900 unknown	0.35 s
Porter fallback	2,900 unknown	2,900 stems	0	0.04 s
Indexing	25,000 tokens	24,700 index entries	300 filtered	0.6 s

10.6 Plot specification. Plot analysis coverage against lexicon size. X-axis: number of stems in the lexicon (50, 100, 200, 300, 500). Y-axis: percentage of annotated word forms receiving a correct analysis. A rising curve that gradually levels off indicates diminishing returns from adding more stems. Also plot runtime against record length to verify approximately linear processing for deterministic regex and DFA stages.

10.7 Reproducibility. Keep the synthetic corpus generator, lexicon CSV, annotation file, source code, test cases and result CSV in one version-controlled folder. Fix the random seed for synthetic data. Never include names, phone numbers or real patient IDs. A report should include the exact command used and the generated output file names.

10.8 Limitations of starter code. The short implementation is a transparent teaching prototype, not a complete production FST. It demonstrates the pipeline interfaces and simple rules. A full submission must expand the lexicon to 300 stems, add all affix classes, complete the four automata, implement formal subset construction and calculate metrics from annotated data. The report should state this honestly.

TASK 11 – COMPARE, RECOMMEND AND DEFEND

11.1 Three strategies. No normalisation is simplest and usually has high exact-match precision, but it misses plurals, tense variants, spelling variants and abbreviations. Porter stemming increases recall with low resource cost, but can over-stem and produce non-words. Full finite-state morphology provides lemma and feature analyses, handles irregular forms through a lexicon and supports generation, but requires expert maintenance.

Criterion	No normalisation	Porter	FST morphology
Retrieval quality	Low recall for variants	Higher recall; variable precision	Best balance for covered vocabulary
Coverage	Only exact forms	Broad English suffix coverage	High for lexicon and productive rules
Runtime	Very low	Very low	Low after compilation; lexicon lookup
Maintenance	Low	Low	High: lexicon, affixes, exceptions
Explainability	Exact string match	Suffix rule trace	Lemma/features and rule trace
Clinical risk	Missed relevant cases	Unsafe conflation possible	Lower when curated; unknown words remain

11.2 Recommended design. Deploy a hybrid architecture: exact regex/DFA validation for structured entities; tokenizer and conservative normalisation; finite-state morphology for approved clinical vocabulary; curated synonym and concept links for derivational relatives; Porter fallback for unknown English tokens; and a review queue for ambiguous or high-risk candidates.

11.3 Expected accuracy and failure modes. Accuracy must be stated from measured evaluation, not assumed. The design should report entity-level precision/recall, morphology coverage and query-level retrieval precision/recall/F1. Likely failures include unknown drug names, misspellings, abbreviation ambiguity, out-of-lexicon words, OCR substitutions, non-English tokens and semantic relations not represented in the terminology map. A clinician may notice a missed case, an irrelevant result, or a result whose matching term has the wrong clinical sense.

TASK 11 – EFFORT, COST AND DEPLOYMENT

11.4 Manual effort. Pattern writing requires a domain expert to define local formats and test edge cases. Lexicon building requires clinical vocabulary selection, part-of-speech labels, continuation classes and irregular forms. Annotation requires at least 200 word forms with lemma/features and 40 queries with relevance judgements. Error review is recurring because hospital abbreviations and documentation habits change.

Work item	Initial effort estimate	Maintenance need
Entity pattern inventory	1–2 engineer days plus clinician review	Review when forms or systems change
300-stem lexicon	2–5 person-days depending on source quality	Add new terms and audit changes
Affix and irregular list	1–3 person-days	Review new terminology
200-form annotation	Several hours to multiple days	Periodic quality checks
40 query judgements	Clinician time; often the main bottleneck	Refresh as search use changes
Testing and release	2–4 engineer days	Regression test each release

These are planning estimates, not measured labour logs. Record actual person-hours by role. A learned approach becomes more economical when the vocabulary and input variation grow faster than experts can maintain rules, and when enough representative labelled data exists. However, high-risk structured fields should retain deterministic validation and human oversight even if learned components are added.

11.5 Final recommendation. Keep the finite-state layer as the explainable core. Add statistical ranking only where ambiguity or noisy input requires it. The goal is not to replace clinical reasoning but to make relevant records easier to find while preserving traceability.

TASK 12 – DAY-4 DESIGN CHALLENGE: IMPACT ANALYSIS

12.1 Changed inputs. The new source includes OCR from handwritten forms and Romanised regional-language notes mixed with English. OCR may confuse 0/O, 1/l and rn/m; split words across line endings; omit diacritics; or insert punctuation. Code-mixed notes may contain regional-language grammatical markers and vocabulary written in Latin characters. At least 20% of tokens may be out of lexicon, misspelt or non-English.

12.2 Broken assumptions. Clean typed input is no longer guaranteed. A closed English vocabulary is unrealistic. English-only morphology cannot analyse regional-language affixes. Exact-match automata have no “near match” state: a single OCR substitution can turn a valid identifier into a rejected string. Regex patterns can also split a hyphenated line-break word incorrectly.

New property	Original component affected	Likely consequence
0/O and 1/l confusion	ID/date/entity regex and DFA	False negatives or incorrect candidate
rn/m confusion	Tokenizer and lexicon lookup	Unknown token
Line-end hyphenation	Tokenizer and word analyser	One word split into two fragments
Lost diacritics	Lexicon and language ID	Missed regional word
Romanised code-mixing	English FST	Low analyser coverage
20% unknown/noisy tokens	Closed lexicon assumption	Coverage and retrieval recall decline

12.3 Precision and coverage estimate. If one fifth of tokens are noisy or out of lexicon, an English-only analyser cannot be expected to analyse all tokens. Coverage may fall by at least the proportion of affected tokens if none can be recovered; actual loss can be higher because noise may damage neighbouring boundaries. Precision can also fall if approximate matching is too permissive. The effect must be measured on a labelled noisy sample rather than inferred as a fixed percentage.

TASK 12 – ORIGINAL DESIGN ON NOISY RECORDS

12.4 Trace A: OCR record. Raw OCR: “Pat1ent AB12O456 adm1tted 05/11/2O26. Fracture of wr1st. Paracetamol 5OOmg.” First failure: patient ID regex rejects `AB12O456` because O is not a digit. The date pattern rejects or misreads the year containing O. The drug-strength pattern may reject `5OOmg`. A global substitution O→0 is unsafe because it could corrupt ordinary words; correction must be context-sensitive.

12.5 Trace B: code-mixed record. Input: “patient ku fever irundhuchu, paracetamol 500mg kuduthen”. English tokenizer recognises patient, fever and paracetamol, but `ku`, `irundhuchu` and `kuduthen` are out of the English lexicon. The English FST returns no analysis. The abbreviation dictionary cannot translate the regional-language grammar. A clinician may still retrieve the record through the English anchors, but the full meaning is not represented.

12.6 Trace C: both OCR and code-mixed. Input: “pat1ent ku fev er irundhuchu; paracetamol 5OO mg kuduthen”. OCR splits fever and corrupts patient and dose. Code-mixed morphology remains unknown. The original exact pipeline loses the patient entity and drug strength, tokenises `fev` and `er` separately, and cannot analyse the regional verbs.

12.7 Alternative formalisms. Weighted finite-state transducers can assign costs to likely OCR corrections. Edit-distance matching can find candidates within a small number of character edits. Character n-gram language identification can estimate whether a token or span belongs to English or a regional language. These approaches are more economical than adding a separate exact regex for every OCR error, but they need carefully chosen thresholds and validation.

TASK 12 – ADAPTATION OR REDESIGN

12.8 Decision: MODIFY. Retain the finite-state core because structured identifiers, known clinical morphology and orthographic rules remain useful. Add a noise-tolerant front end and multilingual routing rather than replacing the entire system.

Layer	Change	Purpose
OCR cleanup	Line-break repair; contextual O/0, l/1 and rn/m candidate generation	Recover likely text without silently altering source
Weighted matching	Weighted edit-distance transducer for selected entity types	Rank near matches
Patterns and automata	Keep exact validators; add candidate mode with confidence	Separate exact acceptance from fuzzy suggestion
Lexicon	Add approved regional-language stems and Romanised variants	Increase code-mixed coverage
Language ID	Character n-gram classifier at token/span level	Route tokens to language-specific analyser
Rule cascade	Add language-specific orthographic rules and aliases	Handle morphology beyond English
Fallback	Porter only for likely English unknown tokens	Avoid applying English suffix rules to every token
Review	Clinician review for ambiguous IDs, drugs and doses	Prevent unsafe auto-correction

12.9 Exact changes. Patterns retain their original strict form for high-confidence extraction. A second fuzzy candidate layer may propose alternatives but must not mark them as validated. Automata gain a candidate-scoring wrapper; the underlying DFA remains deterministic. The lexicon gains regional stems, alternate Roman spellings and source labels. The rule cascade gains language-specific modules selected by language identification. Every correction records original text, candidate, edit operations, weight and approval status.

TASK 12 – QUANTIFIED EFFECT AND DEFENCE

12.10 Evaluation design. Prepare at least three noisy records as described above and preferably a larger labelled set. Compare clean-corpus baseline and noisy-input performance using the same gold annotation policy. Report entity precision/recall, FST coverage, unknown-token rate and query retrieval F1. Measure false correction rate separately; a fuzzy system that increases recall while corrupting doses is unacceptable.

Measure	**Clean baseline (illustrative)**	**Noisy original (illustrative)**	**Modified system target (illustrative)**
Entity precision	0.96	0.78	0.93
Entity recall	0.94	0.62	0.88
Morphology coverage	0.88	0.68	0.82
Retrieval F1	0.87	0.66	0.82
Unsafe auto-correction rate	0.00	0.00	<0.01; require review

These values are target-style illustrative figures, not experimental results. The modified system should be accepted only if measured precision remains above the hospital's safety threshold and no critical identifier or medication field is silently changed. Report confidence intervals if the noisy test set is large enough.

12.11 Evidence-based defence. The original system performs well on clean typed English because its assumptions match its input. The Day-4 requirement violates those assumptions. Exact patterns fail on single-character substitutions; the English lexicon fails on code-mixed morphology; and tokenisation fails on OCR line breaks. A weighted candidate layer addresses character noise, language identification routes code-mixed tokens, and lexicon extension addresses recurring regional forms. The finite-state core remains valuable for deterministic analysis once text is normalised.

TASK 12 – REMAINING LIMITATIONS AND FINAL CONCLUSION

12.12 What finite-state methods will not solve. Even a very large finite-state layer cannot, by itself, reliably interpret arbitrary clinical intent, resolve all semantic ambiguity, infer whether a diagnosis is confirmed or merely suspected, understand discourse across a long record, or replace clinician judgement. It also cannot guarantee correction of severely damaged handwriting when multiple candidate readings are equally plausible.

12.13 Governance. Keep original OCR and typed text immutable. Display uncertain corrections as suggestions. Require confirmation for patient identifiers, medication names, strengths, routes and frequencies. Version every lexicon and rule. Log changes and provide an override mechanism. Test against new hospital departments and documentation templates before release.

12.14 Final conclusion. The best solution is a modified hybrid normalisation and search layer. Regular expressions and DFAs recognise structured entities; tokenisation and conservative rules prepare text; an FST analyses known clinical morphology in both directions; Porter stemming is a limited fallback; and weighted edit-distance plus language identification supports noisy OCR and Romanised code-mixing. The architecture balances recall, precision, explainability, runtime and maintenance effort. It also acknowledges that the assignment's required empirical figures must be obtained from a documented corpus and clinician-judged queries.

APPENDIX A – SYNTHETIC CORPUS PLAN

A.1 Corpus composition. To meet the assignment minimum, create 500 synthetic de-identified records with at least 25,000 word tokens. A practical split is 150 discharge summaries, 150 prescriptions, 100 operation notes and 100 nursing observations. Each record should have a unique synthetic document ID, record type, date, department and generated text. Do not use real patient details.

Record type	Count	Target average tokens	Approximate total
Discharge summaries	150	70	10,500
Prescriptions	150	35	5,250
Operation notes	100	55	5,500
Nursing observations	100	40	4,000
Total	500		25,250

A.2 Data generation fields. Include patient ID, admission/discharge dates, diagnoses, drug-strength expressions, dosage frequency, laboratory values, BP, temperature, ICD code, ward/bed, contact number (synthetic reserved values only), age and symptom duration. Vary spaces, punctuation and case in controlled ways. Store a generation seed and record template version.

APPENDIX B – LEXICON AND ANNOTATION SCHEMA

B.1 Lexicon schema. Required columns: lemma, part_of_speech, continuation_class, clinical_domain, inflectional_features, derivational_relations, irregular_forms, source, reviewer, review_status and version. A 300-stem lexicon should include common diagnoses, anatomy, procedures, symptoms, laboratory terms, medication-related verbs, measurements, ward terms and general clinical verbs.

B.2 Annotation schema. For each of at least 200 word forms, record document ID, token offset, surface form, lemma, part of speech, inflectional features, derivational segmentation, ambiguity alternatives, annotator ID and adjudicated label. Double-annotate a subset and resolve disagreements.

Surface	**Lemma**	**POS**	**Features**	**Segmentation / note**
fractures	fracture	N	PL	fracture + s
treated	treat	V	PAST	treat + ed
bleeding	bleed	V	PROG	irregular stem + ing
diagnoses	diagnosis	N	PL	irregular plural
hypertension	hypertension	N	SG	hyper- + tension; lexicalised
nephrectomy	nephrectomy	N	SG	nephro- + -ectomy; procedure

APPENDIX C – QUERY RELEVANCE JUDGEMENT

C.1 Query set. Create at least 40 clinician queries. Include exact-form queries, inflectional variants, abbreviations, derivational relatives, spelling variants, measurements, date/ID searches and negation-sensitive examples. Each query should be judged against a pool of candidate records by a clinician or authorised domain reviewer.

C.2 Relevance scale. Use binary relevance for the core precision/recall calculation: 1 = relevant, 0 = not relevant. Optionally add graded relevance (0 not relevant, 1 partially relevant, 2 highly relevant) for ranking evaluation. Record why a document is relevant and whether the match is exact, inflectional, derivational, synonym-based or fuzzy.

Query type	Example query	Expected retrieval behaviour
Inflection	fracture	Find fracture, fractures, fractured where context supports
Abbreviation	HTN	Find approved hypertension expansion
Measurement	BP 120/80	Find matching BP observations
Procedure	nephrectomy	Find exact procedure and plural forms
Negation-sensitive	no fracture	Do not treat as confirmed fracture
OCR variant	AB12O456	Suggest candidate only; require verification

APPENDIX D – EVALUATION FORMULAS

Metric	Formula	Interpretation
Precision	TP / (TP + FP)	Fraction of retrieved/matched items that are correct
Recall	TP / (TP + FN)	Fraction of gold items successfully retrieved/matched
F1	2 × Precision × Recall / (Precision + Recall)	Harmonic mean of precision and recall
Coverage	Analysed tokens / eligible tokens	How much vocabulary receives a morphological analysis
Unknown rate	Unknown tokens / all tokens	Proportion not handled by the lexicon/rules
Type-to-lemma ratio	Unique surface types / unique lemmas	Degree of form variation
Runtime	End time − start time	Processing cost under recorded environment
Over-stemming rate	Incorrectly conflated forms / annotated forms	Unsafe merging rate
Under-stemming rate	Related forms left separate / annotated forms	Missed conflation rate

Always define the evaluation unit. Entity precision may be span-level; retrieval precision may be document-level; morphology coverage may be token-level. Do not compare scores with different units as if they were the same measure.

APPENDIX E – IMPLEMENTATION CHECKLIST

- Create 500 synthetic records and verify at least 25,000 word tokens.
- Cover all four required record types.
- Document the 12 listed entity categories (include admission and discharge dates separately).
- Build four automata with at least six states each and include one NFA with epsilon transitions.
- Complete subset construction and DFA minimisation tables from the implemented graph.
- Build at least 300 stems, 20 inflectional affixes and 15 derivational affixes.
- List irregular forms separately.
- Annotate at least 200 word forms and 40 clinician queries.
- Run the three retrieval conditions and compute precision, recall and F1.
- Record TP, FP and FN for every entity pattern.
- Run all test cases and save logs and plots.
- Replace illustrative values in this draft with actual experimental measurements.
- Obtain clinical review before using any normalisation rule on real patient records.

APPENDIX F – REFERENCES AND TECHNICAL SOURCES

Jurafsky, D. and Martin, J. H. (2023). Speech and Language Processing, 3rd ed. draft. Chapters on regular expressions, finite-state morphology and information retrieval.

Porter, M. F. (1980). An algorithm for suffix stripping. Program, 14(3), 130–137.

Koskenniemi, K. (1983). Two-Level Morphology: A General Computational Model for Word-Form Recognition and Production. University of Helsinki.

Hopcroft, J. E., Motwani, R. and Ullman, J. D. (2006). Introduction to Automata Theory, Languages, and Computation, 3rd ed. Pearson.

Manning, C. D., Raghavan, P. and Schütze, H. (2008). Introduction to Information Retrieval. Cambridge University Press.

Bird, S., Klein, E. and Loper, E. (2009). Natural Language Processing with Python. O'Reilly Media.

NLTK Project. Natural Language Toolkit documentation: stemming, tokenization and text processing. https://www.nltk.org/

Python Software Foundation. Python 3 documentation: re — regular expression operations. https://docs.python.org/3/library/re.html

Unicode Consortium. The Unicode Standard. https://www.unicode.org/standard/standard.html

World Health Organization. International Classification of Diseases (ICD) resources. https://www.who.int/standards/classifications/classification-of-diseases

Note: Consult the current edition/version of each online resource when preparing the final bibliography. The assignment PDF itself supplies the hospital scenario and required task structure; the references above support the underlying computational linguistics methods.